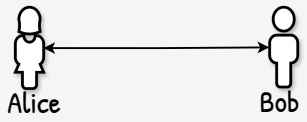
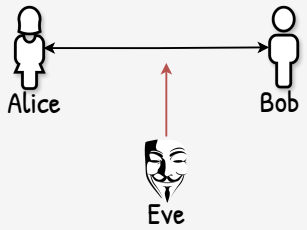
Cryptography I:
Stream Cipher, KDF, PRNG, MAC and Secure channels

Taller de Seguridad

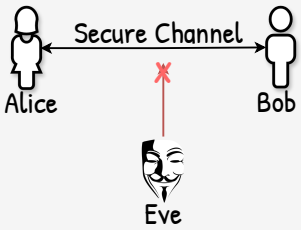
Ing. Martin Di Paola - Fiuba



El escenario es simple, Alice y Bob quieren tener una comunicación.

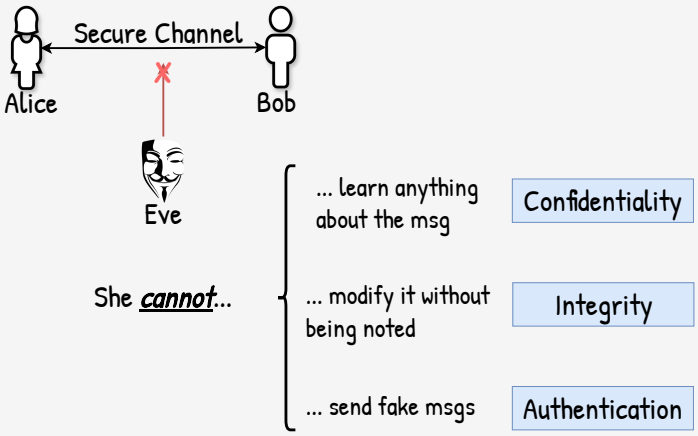


Pero el canal esta abierto y cualquier adversario (como Eve) puede acceder a él.



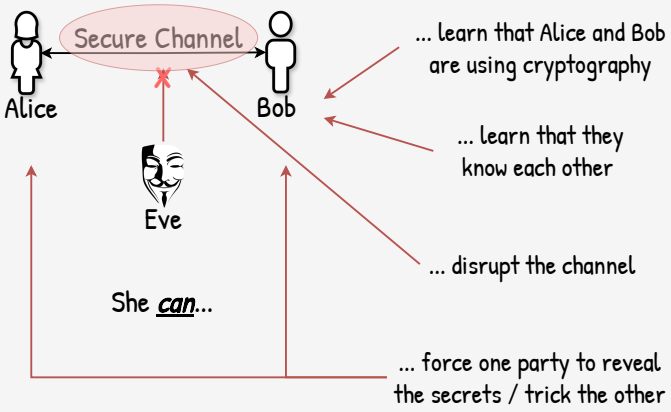
La premisa es simple: aun estando en un escenario abierto, se quiere construir un canal seguro entre Alice y Bob para q ningun adversario pueda ver de que estan hablando.

Veamos exactamente q queremos decir con "canal seguro".



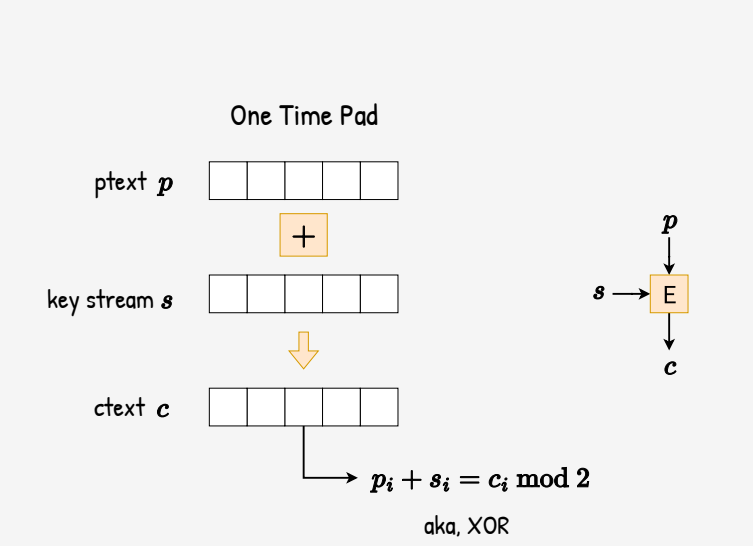
Hay 3 propiedades que queremos tener:

- confidencialidad: Eve no puede saber de q estan hablando Alice y Bob
  - En particular, no puede deducir absolutamente nada.
  - Ni siquiera si un bit es o no más probable que sea un 0 q un 1.
- integridad: Eve puede modificar el o los mensajes, incluso puede cambiarlos de orden, duplicarlos o borrarlos; lo que queremos es que si Eve hace algo asi, que sea trivial de detectarlo para Alice y para Bob (cosa de que aborten la comunicación)
- autenticidad: aunque Eve puede crear mensajes, q no pueda engañar a Bob diciendo q vienen de Alice (y viceversa); Eve no puede falsificar ni hablar a nombre de otro. Si lo hace, debe ser trivial detectarlo para Alice y para Bob.



Pero un canal seguro no es perfecto. Eve tiene bastante control sobre la comunicación:

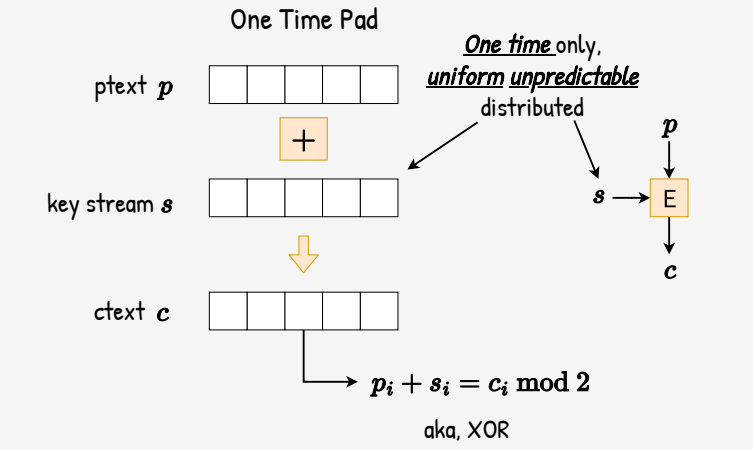
- no sabe nada del mensaje, pero sabe q Alice y Bob se estan comunicando, que deben conocerse, sabe a que hora, por cuanto tiempo y que estan usando criptografía.
- no pude alterar los mensajes ni falsificarlos, pero puede disrumpir/bloquear la comunicación (pone un firewall por ejemplo).
- y en última instancia, Eve puede ir a la casa de Alice y forzarla a revelar toda la comunicación y/o a enviar mensajes a Bob a nombre de ella.



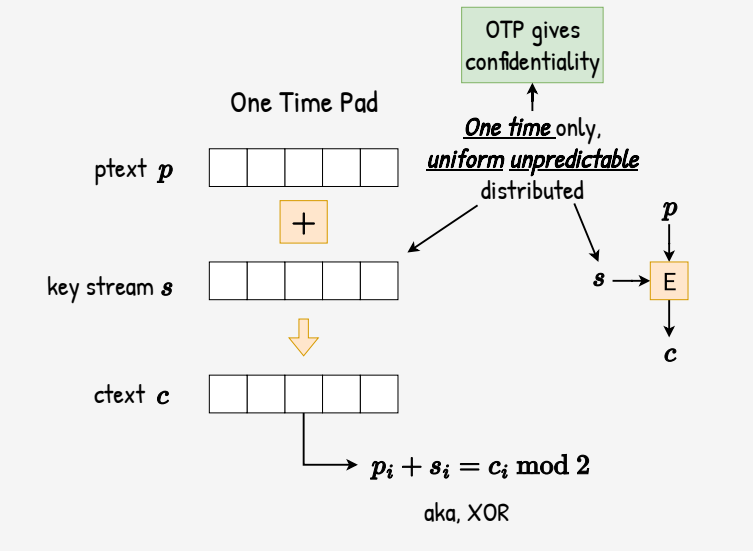
Dado un plaintext se aplica xor bit a bit con un key stream para dar lugar a un ciphertext.

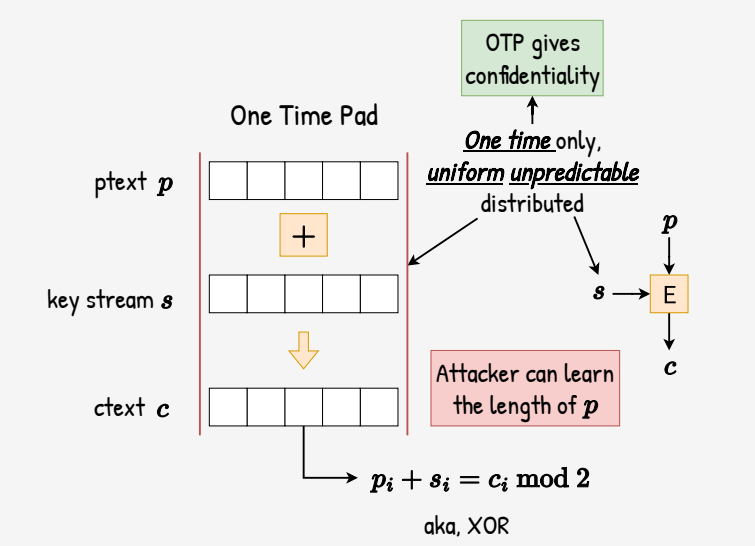
El proceso de tomar un plaintext y generar un ciphertext se llama encriptación o cifrado. Debe ser una operación reversible para que el receptor del ciphertext pueda obtener nuevamente el plaintext.

OTP es reversible, basta con realizar un segundo xor con el mismo key stream para cancelar al primero.

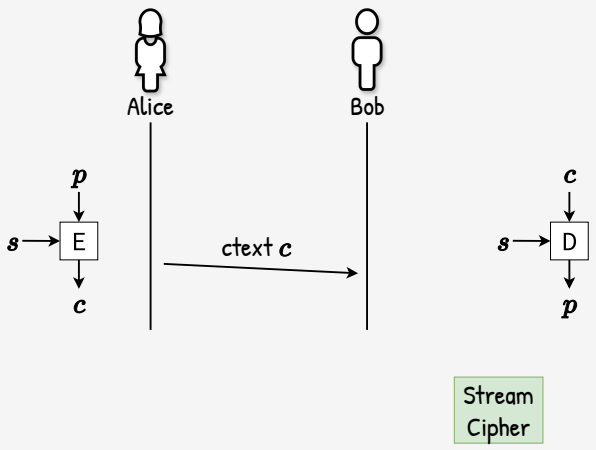


Si el key stream es totalmente aleatorio y usado una unica vez, no es posible aprender nada del plaintext a partir del ciphertext. OTP ofrece confidencialidad.



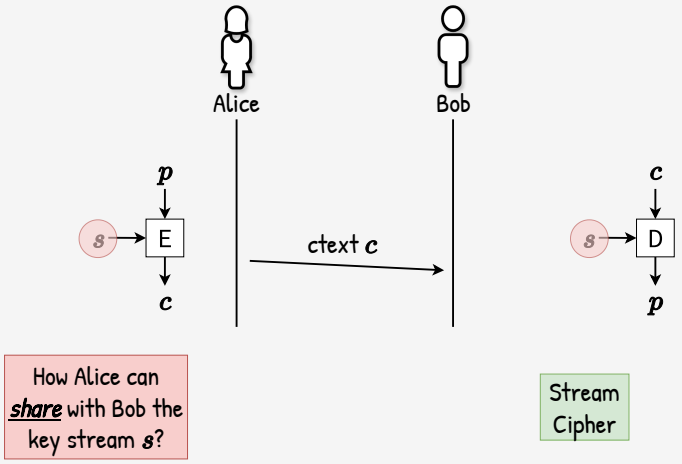


Pero OTP tiene puntos débiles: el adversario puede saber que tan largo era el plaintext original.



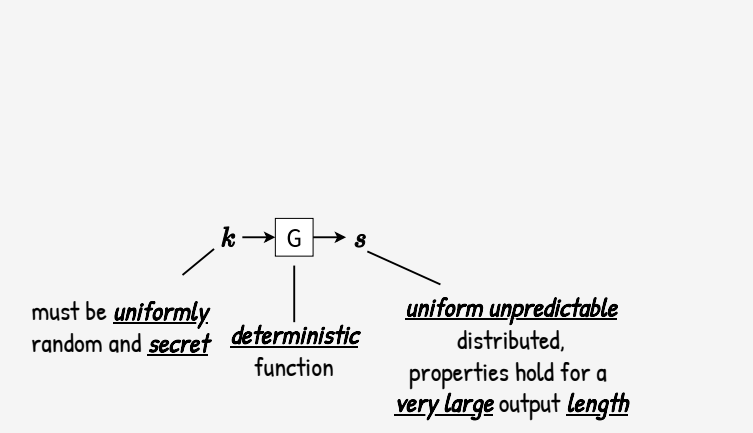
Entonces, armados con un OTP, Alice encripta el mensaje, le envia el ciphertext a Bob y este lo desencripta.

El canal es confidencial: Eve no puede saber nada del mensaje sin haber obtenido/robado el key stream.



Pero como hacen Alice y Bob para tener exactamente el mismo key stream?

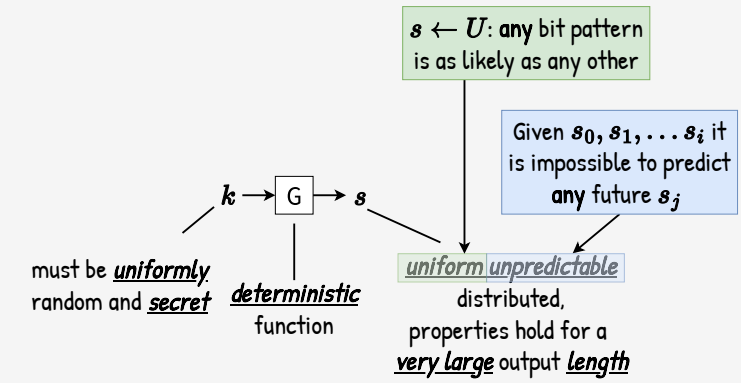
Si fuese trivial y seguro compartirlo por algun medio, por que no usar ese mismo medio para hablar de forma segura directamente?



Lo q necesitamos es una forma de generar secuencias aleatorias grandes a partir de una entrada más chica, fácilmenten compartible de forma segura.

El CSPRNG es una función determinística que a partir de una clave genera una secuencia larga de bytes. Si la clave es uniformemente distribuida, un CSPRNG debería generar una secuencia uniformemente distribuida, totalmente impredecible sin saber la clave.

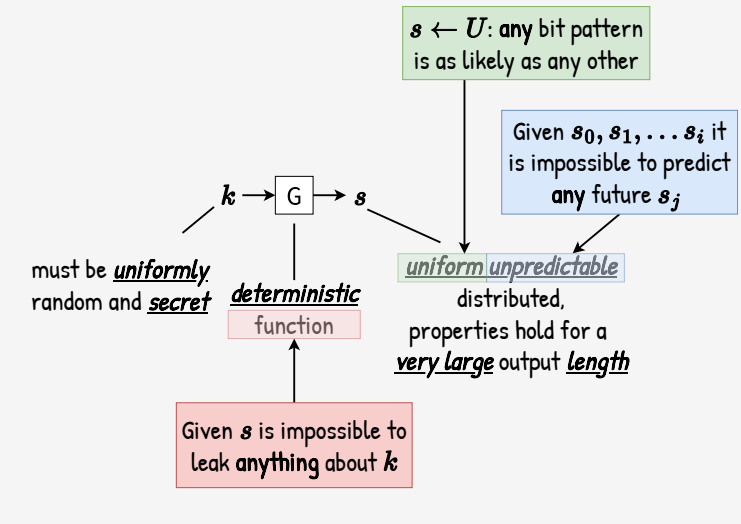
Todo CSPRNG tiene limitaciones, y pueden mantener sus propiedades de uniformidad solo hasta cierta cantidad de bytes generados (un cientos de megabytes). Luego, el output puede empezar a presentar repeticiones o sesgos q lo hacen inusable para aplicaciones criptográficas.



Con uniformemente distribuido decimos que cada bit es o un 0 o un 1 con 50% de probable cada uno; que cada secuncia de 2 bits, 00, 01, 10, 11 tiene un 25% de probabilidades; que cada secuncia de 3 bits ....

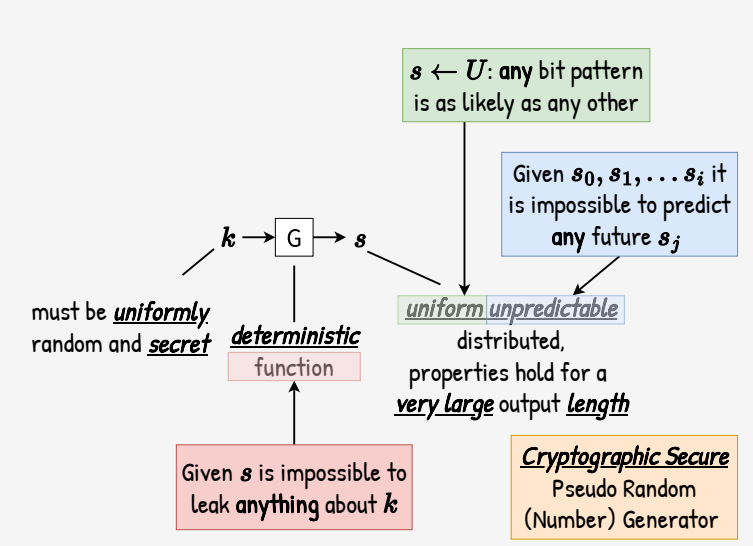
En general, que cualquier patrón de bits es tan probable como cualquier otro.

Y con impredecible decimos que si un adversario sabe todos los bytes generados por el CSPRNG, no puede predecir que valor tendra ningun bit futuro.



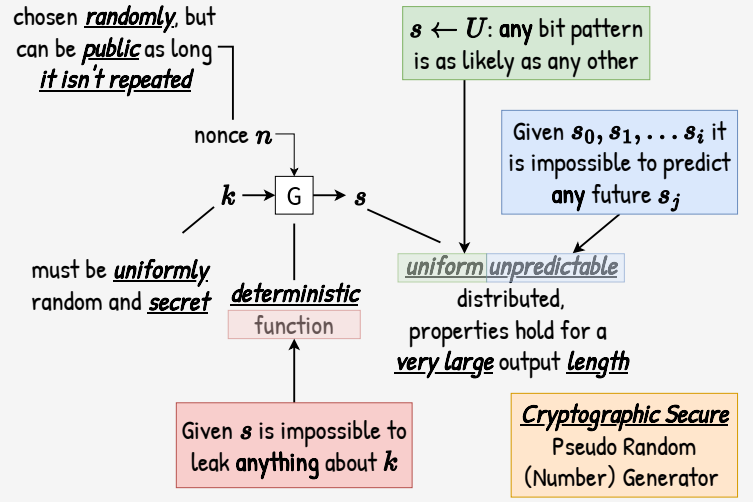
Con funcion nos referimos a la defincion matematica: un mapeo del espacio de claves al espacio de key streams.

En particular, dado la salida del CSPRNG, no es posible aprender nada de la clave secreta usada como entrada/input.



Existen múltiples generadores de numeros pseudo aleatorios (PRNG), pero solo aquellos que cumplen con las condiciones de uniformidad, imprevisibilidad y no reversibilidad son criptográficamente seguros.

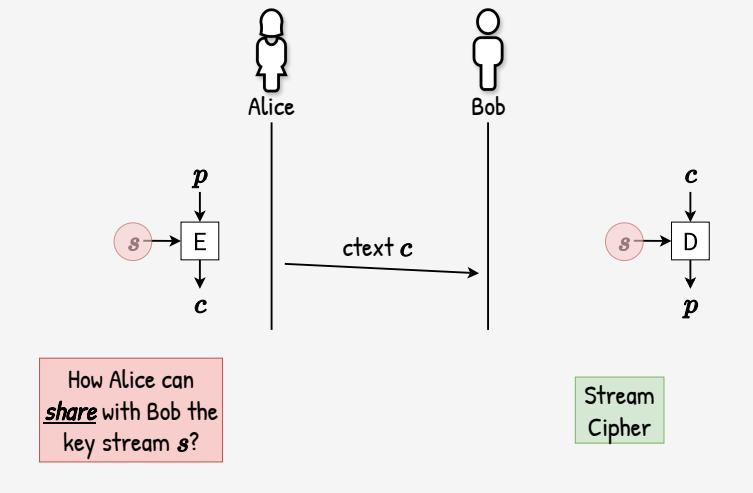
Usar un PRNG en vez de un CSPRNG, deja vulnerable al cipher y un atacante puede terminar rompiendo todo el sistema.



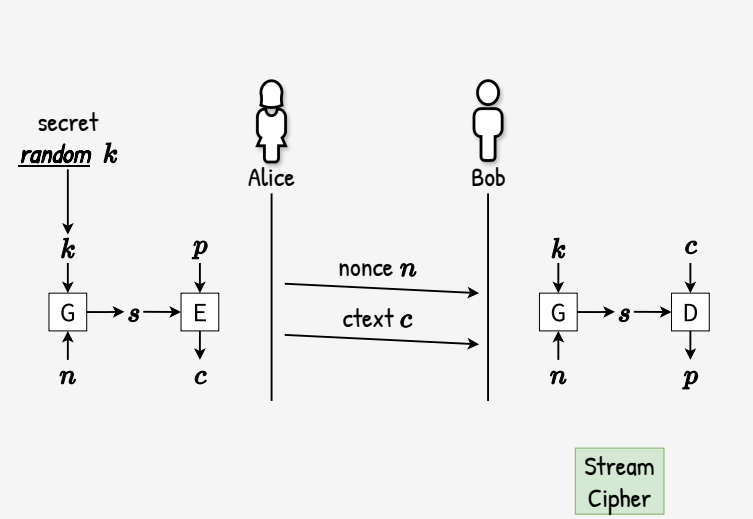
Pero si el CSPRNG solo puede generar hasta cierta cantidad de bytes antes de perder las propiedades estadisticas, como podemos usarlo para generar un key stream mas largo?

En general los CSPRNG tienen alguna manera de agregarle un nonce, una entrada/input adicional que, si es aleatoria y no se repite, hace que con cada nuevo nonce se obtenga una nueva secuencia perfectamente aleatoria.

Con ellas, se puede armar un key stream lo suficientemente largo.



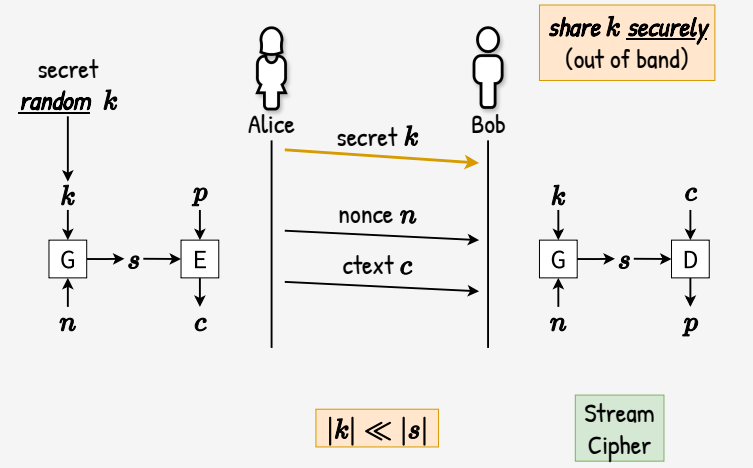
.



Aca entra en juego el CSPRNG.

Alice genera una clave secreta totalmente aleatoria y se la pasa a Bob de forma segura. Ambos luego usan un CSPRNG para generar la misma sequencia larga de bytes aleatorios que puede usarse como key stream.

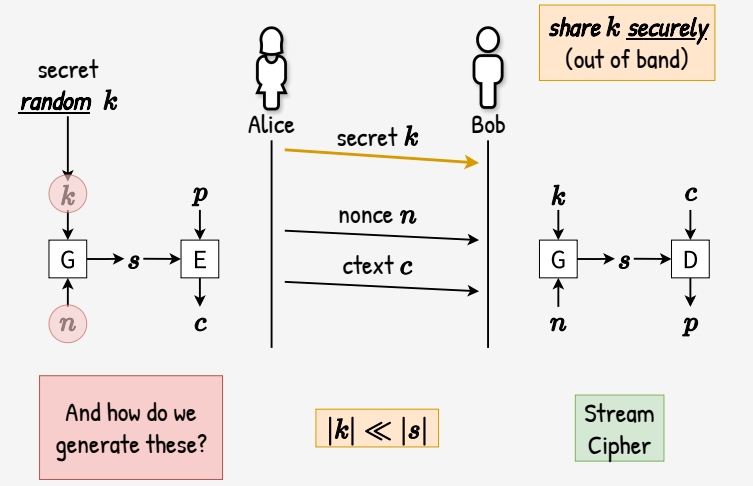
La combinación del OTP (xor) y del CSPRNG forman un Stream Cipher. A menudo, el combo se lo llama encriptador / cipher pero en rigor son 2 partes separables, el OTP y el CSPRNG.



Y como se comparte la clave secreta? Vamos a asumir por el momento que Alice puede pasarsela a Bob de manera segura.

Es algo de trampa pero perfectamente real. A diferencia de pasarse el key stream, pasarse la clave secreta es factible.

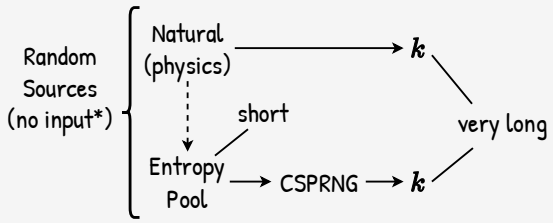
No hablamos de megabytes/gigabytes, hablamos de pasarse unos pocos bits, 128 o 256. Es perfectamente factible q Alice se lo pase a Bob por papel cuando se juntaron en un café.



Y como puede Alice generar una clave aleatoria, uniformemente distribuida?

Armado con un CSPRNG, solo falta saber como obtener claves y nonces aleatorios, uniformemente distribuidos.

Tarea no trivial ya que nuestras computadoras son determinísticas!

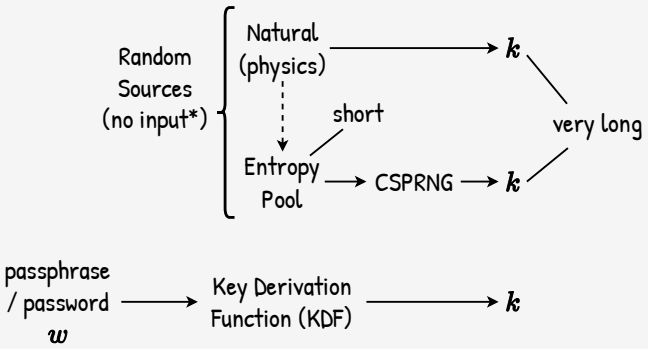


Fuentes de aleatoriedad, hay 2 formas:

- O sacamos bits aleatorios de procesos fisicos fundamentalmente aleatorios
- O obtenemos unos pocos bits aleatorios y usamos otro CSPRNG para

  tener mas bytes aleatorios.

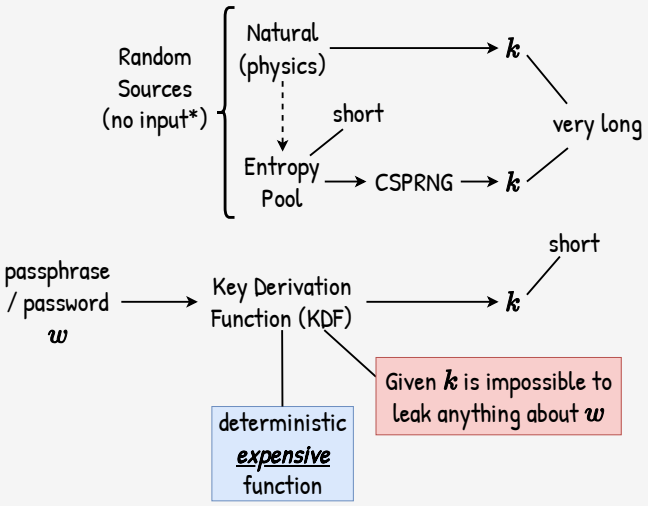
La última forma es medio cíclica, pero tiene sentido: solo necesitamos 256 o 512 bits (o en ese orden) que sean realmente aleatorios para que con un CSPRNG podamos generar una cantidad mucho mayor de bytes aleatorios. El OS hace esto por vos y en Linux, se puede obtener bytes criptográficamente aleatorios leyendo de `/dev/random`.



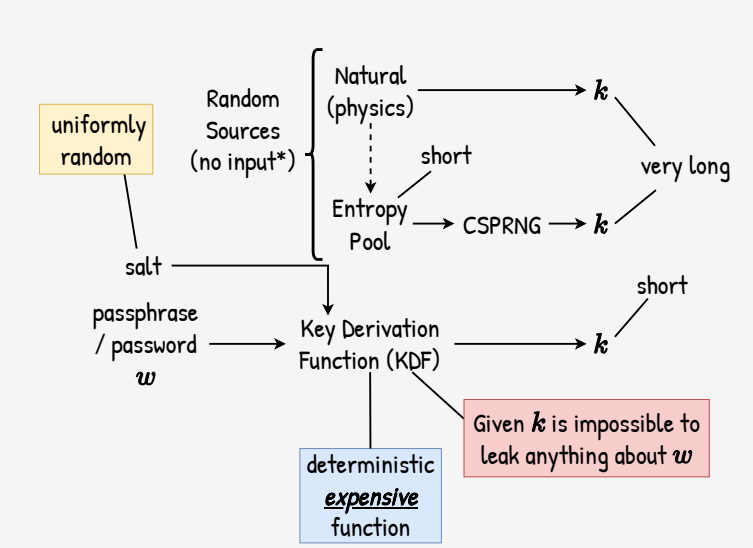
Generar claves y nonces aleatorios de esas formas es razonable para máquinas.

Pero si Alice le quisiera pasar a Bob una clave, suena poco humano estar memorizandose 256 bits aleatorios.

Aqui entran las Key Derivation Functions (KDF): toman una password/passphrase mas humana, que aunque no es aleatoria no debe ser adivinable por un adversario, y de ella se obtiene una clave uniformemente distribuida.



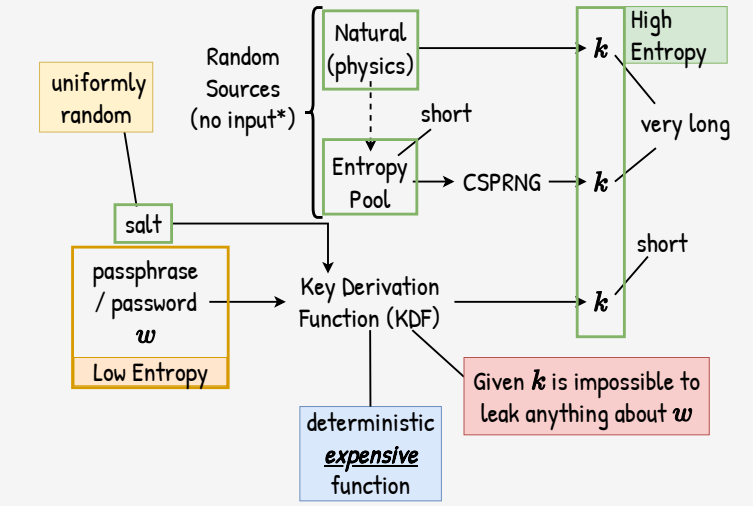
Las KDF son la parte mas sensible del sistema. Deben ser lo mas dificil de computar para que un adversario tenga un costo alto de bruteforcing y deben garantizar que si la clave se leakea, no se puede aprender nada de la password original.



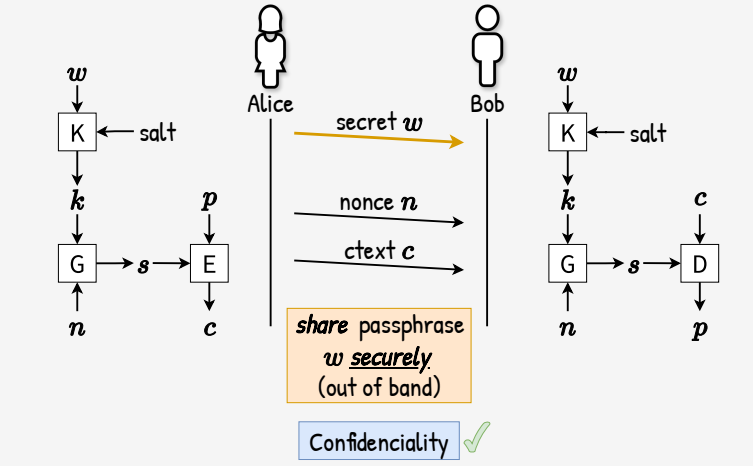
Dado q las KDF son deterministicas, si 2 personas usaran la misma password, generarian la misma clave secreta que, en ciertos contextos, puede destruir la seguridad de una comunicacion.

Por eso las KDF suelen aceptar un parametro adicional llamado salt. Este cumple 2 roles:

- si 2 personas usan el mismo password, como el salt es al azar, van a generar 2 claves diferentes
- dado que el password tiene baja entropia, la salt aumenta la entropia de la entrada del KDF mejorando la aleatoriedad de la salida.

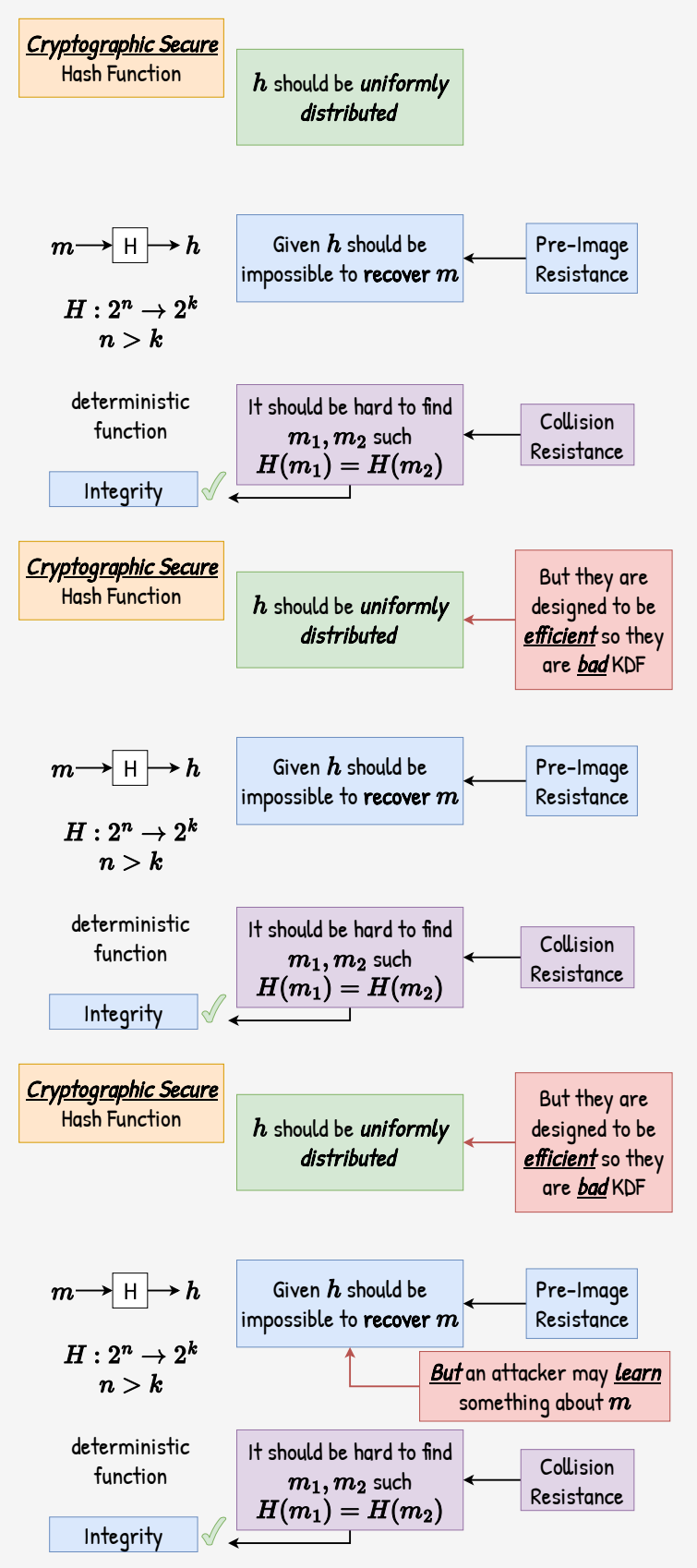


El juego es el mismo: se toma algo de baja entropia (baja aleatoriedad) y se lo pasa a algo de alta entropia, apto para usos criptográficos.



Con el Stream Cipher y con la generacion de la clave y nonce de forma aleatoria, uniformememnte distribuida tenemos lista la confidencialidad.

Por el momento vamos a asumir que Alice puede pasarle a Bob la passphrase de forma segura.



Ahora nos enfocamos en la integridad. Una función de hash criptográficamente segura toma un mensaje y nos da un secuencia fija de bytes aleatorios.

La operación inversa es imposible: dada el hash no es posible obtener el mensaje original.

Y para garantizar integridad, no es posible encontrar 2 mensajes distintos que tenga un mismo hash. En particular, el adversario no puede modificar un mensaje y aun asi obtener el mismo hash q el original.

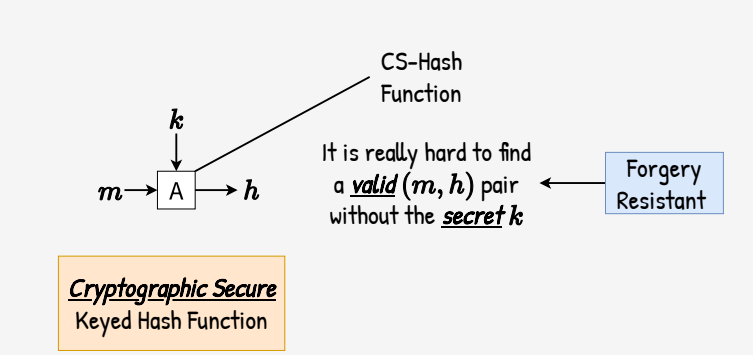
Notar que toda función de hash tiene un dominio (espacio de entrada) mucho más grande que el espacio de salida por lo q esta garantizado que habran colisiones. La idea es q estas sean muy difíciles de encontrar, y muchisimo más aun de fabricarse a voluntad.

Es tentador ver a una función de hash como un KDF pero es mala idea. Las funciones de hash deben ser rápidas de ejecutar; los KDF todo lo contrario.

Una solución de compromiso seria implementar un KDF con hashear muchísimas veces el passphrase, para asi enlentecer el proceso. Aun asi sera cuestión de tiempo hasta q el hardware sea aun más rápido de lo q es hoy y la cantidad de hasheos sea insuficiente.

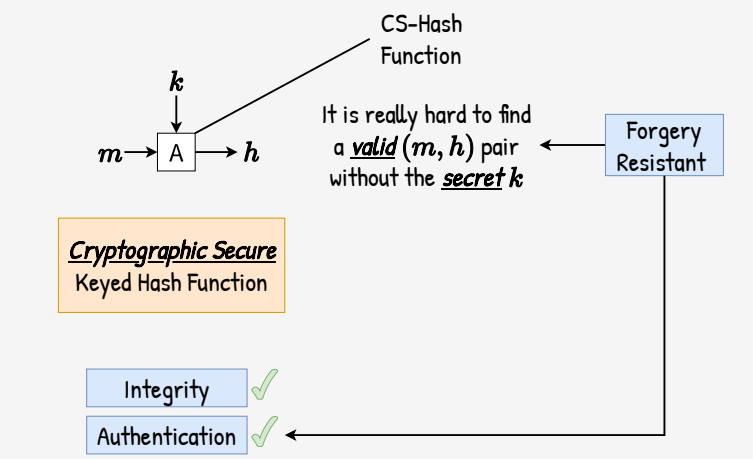
En lo posible, no usar hashes como KDF.

Y ojo, si bien dado un hash uno no puede recuperar el mensaje original, las funciones de hash criptográficamente seguras no necesariamente garantizan que no se leakeen bits del mensaje en el hash.

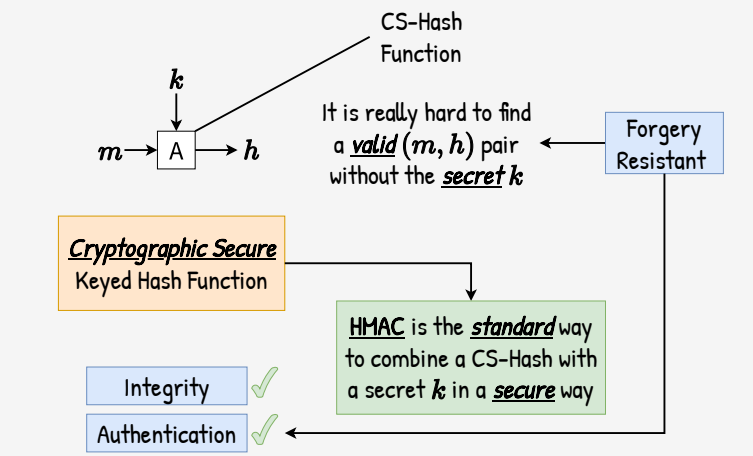


Si introducimos un secreto, podemos forzar a que solo aquellos que conozcan el secreto puedan generar hashes válidos.

Esto es un Message Authentication Code (MAC).

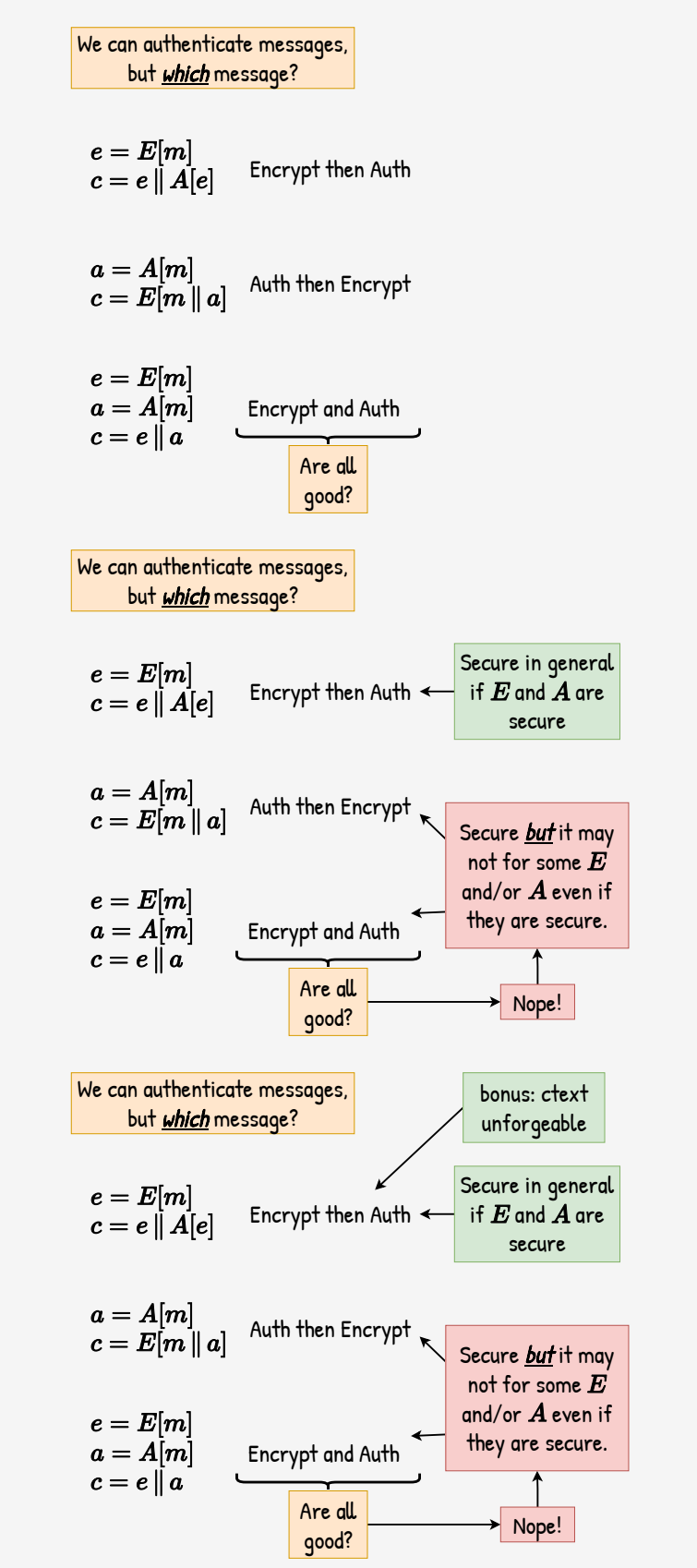


Y con esto tenemos autenticación.



Hay varias formas de introducir/combinar un secreto con el mensaje y en general todas estan rotísimas!

Hoy el estandar es usar HMAC, una forma específica de combinar secretos y mensajes y hashearlos de forma segura. No inventar!



Y exactamente que mensaje queremos autenticar? (MAC sobre que mensaje?)

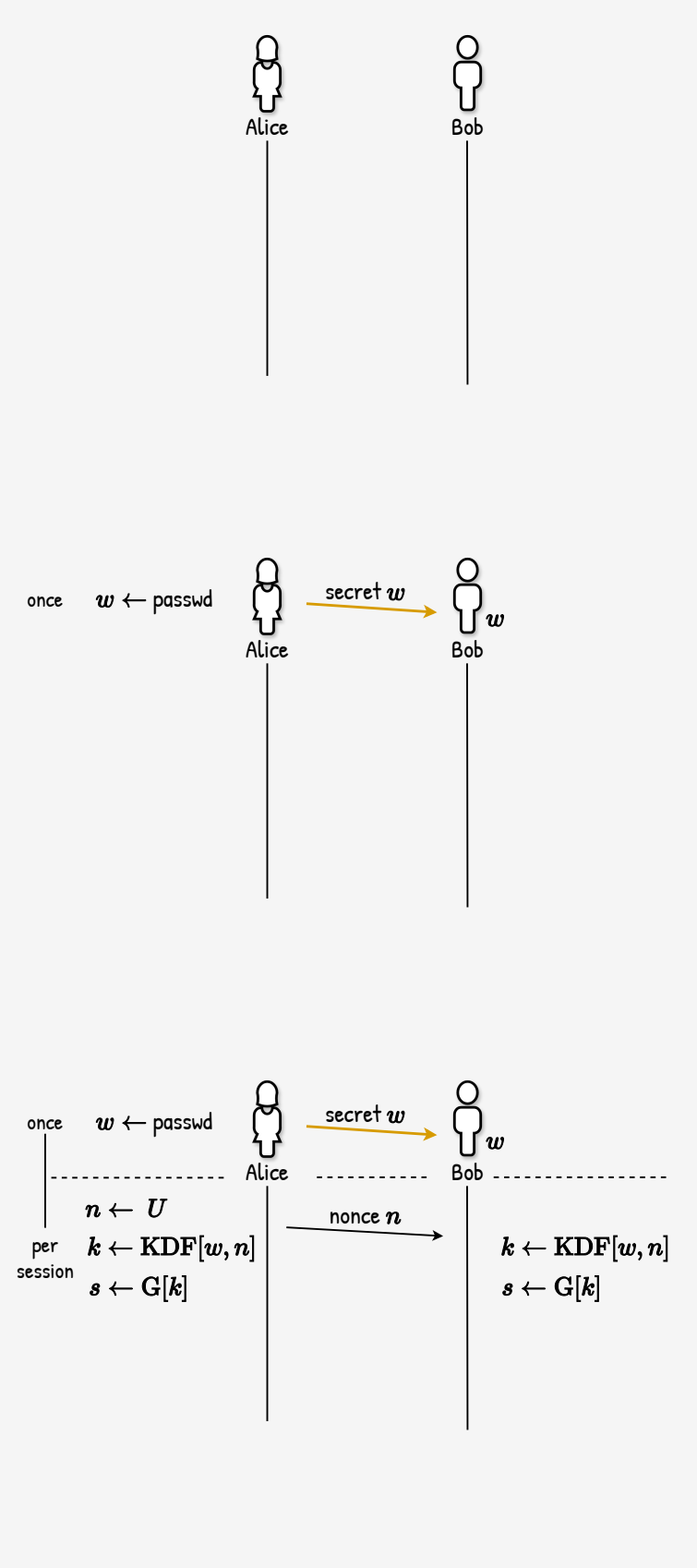
Sobre el plaintext? sobre el ciphertext? La MAC puede ir en plano o debe ir cifrada?

No es una repuesta trivial y muchos protocolos viejos estan rotos por haber elegido mal!

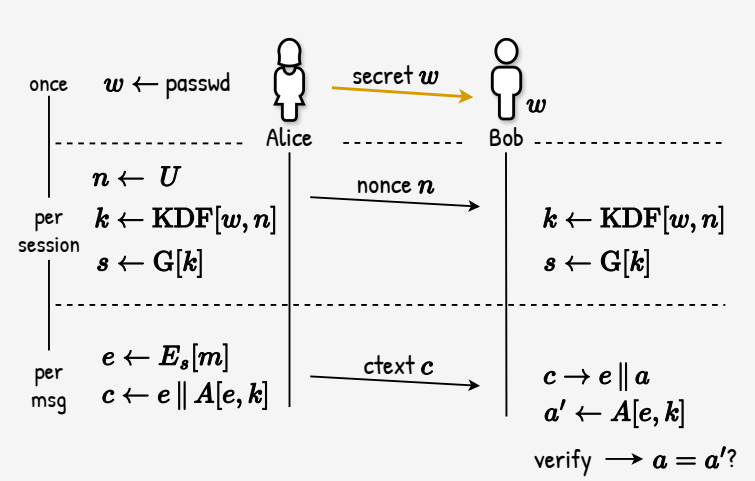
.

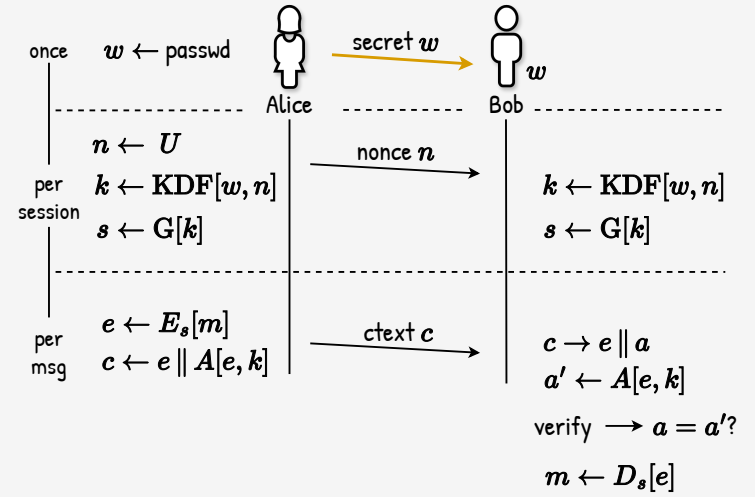
Bonus: Encrypt and then Authenticate hace q el ciphertext resultante no pueda se modificado por un adversario sin que se note.

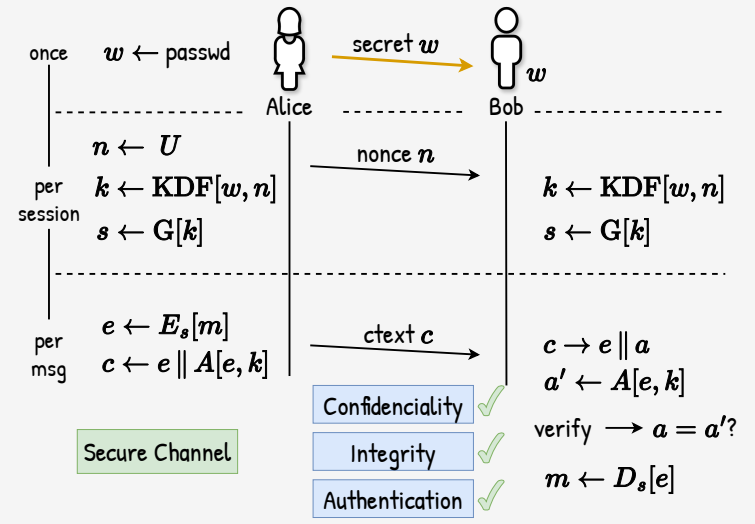
Si modifica la parte q corresponde a la encriptación del plaintext, la verificación del MAC fallara; si modifica la parte de la MAC, la verificación también fallara.


Alice
Bob
once
$w \leftarrow$ passwd
secret w
w
per session
$n \leftarrow U$
$k \leftarrow \mathbf{KDF}[w, n]$
$s \leftarrow \mathbf{G}[k]$
nonce n
$k \leftarrow \mathbf{KDF}[w, n]$
$s \leftarrow \mathbf{G}[k]$

Ready? Go!



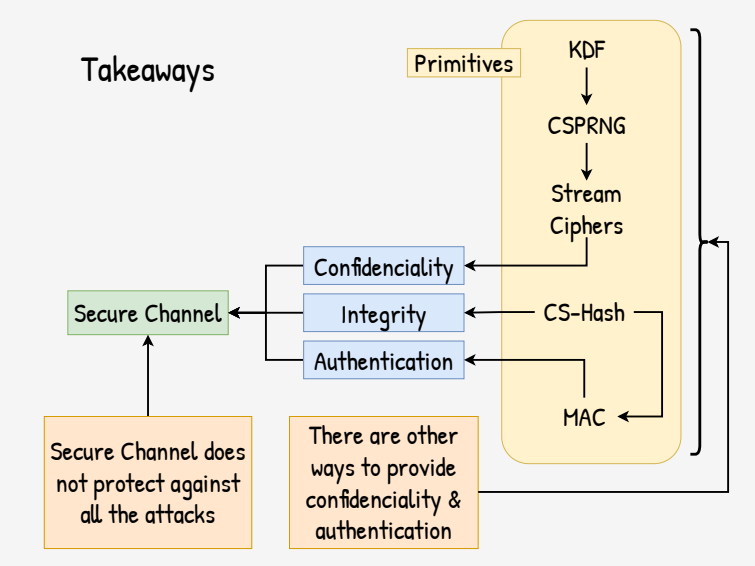




El slide muestra como Alice elige un nonce y de ahi tanto Alice y Bob deducen el mismo key stream. Este es solo para encriptar + MAC mensajes de Alice hacia Bob; Bob no puede usar ese mismo key stream para enviarle mensajes a Alice por q podria inadevertidamente desincronizarse con ella.

Si Bob quiere responderle debe elegir su propio nonce y pasarselo a ella y de ahi, ambos generar un segundo key stream que sera usado solo para mensajes de Bob hacia Alice. (Lease, es el mismo slide pero reflejado).

Si la comunicación es muy larga, Alice y Bob pueden generar nuevos nonces y generar nuevos key streams frescos para q estos no pierdan las propiedades de uniformidad y imprevisibilidad.



Cripto es un universo. Solo vimos la punta del iceberg.